

UNICC AI Safety Lab: Design and Implementation of a Multi-Agent AI Governance Evaluation System

Applied Technical Project

Final Report

by

Tongye Ding

td2124

Spring 2026

A paper submitted in partial fulfillment of the requirements for the degree of

Master of Science

at the

Division of Programs in Business

School of Professional Studies

New York University

Abstract

This report presents the UNICC AI Safety Lab project as an applied technical prototype for AI governance evaluation. The project implemented a multi-agent system that assesses proposed AI use cases across three complementary dimensions: standards alignment, safety risk, and assurance and accountability readiness. The system architecture comprises three FastAPI-based specialist agents, an orchestrator service, YAML-driven configuration, prompt files, shell launch scripts, and a React dashboard for end-user interaction. A local Ollama endpoint using the llama3.2 model was used to support privacy-conscious experimentation and modular deployment. The project addressed the practical organizational problem of inconsistent and opaque AI review processes by introducing a structured evaluation flow that surfaces triggered standards, governance gaps, required controls, risk scores, evidence weaknesses, and a consolidated deployment recommendation. A sample AI hiring screening use case demonstrated the system's behavior in a sensitive domain. The recorded output generated an Approve with Conditions decision and identified concerns related to privacy, accountability, transparency, bias testing, human oversight, and data governance. The project's main contribution is an explainable, extensible architecture that decomposes AI governance reasoning into specialized services and then aggregates the findings into a single review outcome. While the prototype is not a substitute for

formal legal review or enterprise-grade governance operations, it provides a credible foundation for future development into a more robust AI review platform. The report evaluates the project’s problem context, architecture, objectives, risks, results, limitations, and future opportunities, and concludes that the prototype is a meaningful applied demonstration of how a governance-oriented AI safety lab can be designed and implemented.

Table of Contents

Abstract	3
Table of Contents	4
Acknowledgements	6
Declaration	6
Abbreviations and Terms.....	7
1. Introduction	8
2. Company Background and Sponsor Information.....	10
3. Problem Description and Opportunity	12
4. Importance of the Project to the Organization	15
5. Alternative Solutions Evaluated.....	17
6. Solution Evaluation Criteria and Selection Rationale	19
7. Approach and Methodology	21

8. Project Objectives and Metrics for Project Evaluation	23
9. System Architecture and Technical Design	26
10. Detailed Component Analysis	29
11. Demonstration Scenario and Results Interpretation	32
12. Risk Analysis	34
13. Issues Encountered and How They Were Handled	36
14. Detailed Project Chronology and Critique	38
15. Lessons Learned	40
16. Conclusions and Summary	42
17. Limitations, Recommendations, and Scope for Future Work	44
18. Literature Survey	46
19. Situational Analysis	48
20. End-to-End Workflow Analysis	50
21. Data Flow and Information Handling	51
22. Governance Mapping to Recognized Framework Themes	53
23. Testing Strategy and Validation Considerations	54
24. Ethical Critique of the Prototype	56
25. Managerial Implications	58
26. Deployment Roadmap for a Future Version	59
Appendices	61

Appendix A. Project Acceptance Document Placeholder	61
Appendix B. Project Plan, Baseline, and Actual WBS.....	63
Appendix C. References and Bibliography	63
Appendix D. Other Supporting Documents	64

Acknowledgements

Sincere thanks are extended to the Applied Project sponsor and to the faculty guidance that supported the completion of this project. Appreciation is also due to the instructors in the NYU SPS program whose coursework in analytics, management, and applied technology helped shape the analytical and implementation approach reflected in this report. This project also benefited from prior learning in AI governance, data analysis, and full-stack prototyping, all of which informed the final project outcome.

Declaration

I grant powers of discretion to the Department, SPS, and NYU to allow this paper to be copied in part or in whole without further reference to me. This permission covers only copies made for study purposes or for inclusion in Department, SPS, and NYU research publications, subject to normal conditions of acknowledgement.

Abbreviations and Terms

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
FASTAPI	Python web framework used to expose HTTP endpoints
LLM	Large Language Model
UNICC	Project context for the AI Safety Lab initiative
JSON	JavaScript Object Notation
YAML	YAML Ain't Markup Language
UI	User Interface
RMF	Risk Management Framework

1. Introduction

The Applied Project documented in this report focused exclusively on the UNICC AI Safety Lab initiative and its development as a multi-agent AI governance evaluation system. The project was designed as a practical software prototype that demonstrates how an organization can structure AI review into specialized, explainable components rather than relying on a single opaque assessment step. The repository provided for the project shows a working architecture composed of three FastAPI-based specialist agents, a coordinating orchestrator, YAML configuration files, prompt files, shell launch scripts, and a React dashboard. Collectively, these components form a self-contained environment for evaluating proposed AI use cases across standards alignment, safety dimensions, and assurance and accountability readiness.

The sponsoring context for the project was the AI safety and governance problem space addressed through the UNICC AI Safety Lab concept. The technical implementation examined in the repository was not positioned as a production compliance engine that renders legal determinations. Instead, it was intentionally framed as a governance evaluation and readiness support tool. This distinction is important. Formal legal compliance, especially in a rapidly changing area such as artificial intelligence regulation, requires legal review, policy interpretation, documented controls, and organizational accountability structures. The project therefore concentrated on a decision-support

design that helps users identify likely governance triggers, surface missing controls, and generate a practical recommendation such as Approve, Approve with Conditions, Flag for Review, or Reject.

The project's core value proposition lies in decomposing AI governance into specialized viewpoints. One agent evaluates external standards and governance frameworks. A second agent scores the proposed AI system across selected safety dimensions such as harmfulness, bias and fairness, transparency, deception, privacy, and accountability. A third agent assesses the quality of implementation evidence, ownership, monitoring, escalation, and residual deployment risk. An orchestrator then aggregates the outputs into a consolidated decision and cross-agent summary. This architecture improves explainability because each agent is responsible for a distinct reasoning role, and it improves maintainability because agents, prompts, and configurations can be changed independently.

This report presents the project in a professional final-paper format aligned with course requirements. It covers the business problem, project importance, alternative solution patterns, methodology, objectives and metrics, risk analysis, issues encountered, project chronology, lessons learned, conclusions, limitations, future work, literature survey, and situational analysis. The report also includes appendices that support the structure required for an applied project submission. Where sponsor-specific signed documents were not available in the materials provided to this session, clearly labeled placeholders

were included so that the final document can be completed with the required signatures before submission.

The remainder of the paper demonstrates that the UNICC AI Safety Lab project is not merely a conceptual discussion of AI ethics. It is an implemented prototype with traceable software components, a defined evaluation workflow, and a demonstrable use case. The project therefore occupies a useful middle ground between academic AI ethics discussion and enterprise governance tooling. It is technical enough to show system design and implementation, while still managerial enough to address risk, accountability, oversight, and organizational readiness.

2. Company Background and Sponsor Information

The project was framed around the UNICC AI Safety Lab concept. In this report, UNICC is treated as the organizational context in which an AI safety and governance review capability would be valuable. The project repository itself emphasizes an AI governance evaluation workflow rather than a domain-specific operational AI application. That focus suggests that the project's organizational purpose is to support decision makers, reviewers, and governance stakeholders who need a consistent method for examining AI proposals before deployment.

A governance-oriented lab environment is particularly relevant for organizations that oversee multiple projects, jurisdictions, risk domains, or stakeholder groups. In such

environments, AI systems cannot be assessed solely on the basis of model performance. They must also be reviewed for fairness, privacy, transparency, accountability, evidence sufficiency, human oversight design, and controls for downstream monitoring. The UNICC AI Safety Lab concept addresses this need by providing a structured evaluation mechanism that can be applied across use cases. The system is therefore best understood as an internal review tool that helps organizations raise the maturity of AI deployment decisions.

The practical sponsor role in an applied project of this kind would normally include confirming the business need, validating the relevance of the selected governance dimensions, reviewing the quality of the prototype, and determining whether the final output meets organizational expectations. In a completed course submission, this role would be reflected in the signed Sponsor Acceptance Document and the sponsor information section required by the course guidelines. Since the signed documentation was not included in the materials supplied here, this report identifies the sponsor role functionally and leaves final sponsor-specific metadata to be inserted before submission.

The project environment indicated by the repository suggests that the intended users include technical reviewers, policy or compliance reviewers, risk professionals, project sponsors, innovation teams, and possibly instructors or evaluators in an academic demonstration setting. The included sample payload uses an AI hiring screening use case, which is a suitable governance test case because it touches a sensitive domain,

involves human subjects, and raises potential issues of fairness, privacy, transparency, and accountability. The use of such a scenario indicates that the sponsor context values high-impact governance review rather than low-risk experimentation alone.

From a systems perspective, the organizational setting also appears to value local deployment and privacy-conscious experimentation. The project uses Ollama with a local llama3.2 model endpoint rather than a hard-coded dependency on a remote commercial model service. This design choice is consistent with governance and safety objectives because it reduces external dependency and gives the implementing organization more control over where prompts and responses are processed. In summary, the sponsoring environment for the project can be described as a governance-focused, risk-aware setting in which explainable, modular AI review is more valuable than a monolithic or purely performance-oriented solution.

3. Problem Description and Opportunity

The central problem addressed by the project was the lack of a structured, repeatable, and explainable process for evaluating proposed AI use cases before deployment. Many organizations discuss AI safety and ethics at a high level, yet their review processes remain fragmented. Legal, technical, ethical, and operational questions are often raised in separate conversations with inconsistent criteria and incomplete documentation. As a result, organizations may approve systems without sufficient controls, delay systems

because concerns are not well organized, or fail to create a record of how governance decisions were made. The UNICC AI Safety Lab project addressed this challenge by proposing a software-assisted review workflow.

A single reviewer or a single model prompt is often inadequate for governance evaluation because AI governance is multi-dimensional. Standards alignment requires knowledge of frameworks and obligations. Safety evaluation requires attention to possible harms and vulnerable groups. Assurance evaluation requires operational thinking about evidence, monitoring, human review, ownership, and incident response. When these forms of reasoning are compressed into a single undifferentiated assessment, the result can be vague, inconsistent, or difficult to defend. The opportunity, therefore, was to build a modular review mechanism that separates these lenses while still producing one consolidated output.

The repository's structure confirms that the solution opportunity was framed as a multi-agent system. Three specialist agents receive the same core use case payload but interpret it through different prompts and output schemas. The orchestrator then calls each agent, normalizes the results, compiles cross-agent risks, gaps, and controls, and produces a final decision. This approach creates traceability because a reviewer can see which part of the final conclusion came from standards reasoning, which part came from safety scoring, and which part came from assurance analysis. For organizations

attempting to improve AI governance maturity, such traceability is a significant advantage.

The project also addressed the opportunity to make governance evaluation more accessible to non-specialist stakeholders. The included React dashboard provides a user interface where a reviewer can enter a title, description, intended users, affected groups, data sources, deployment context, current controls, geography, and high-level contextual toggles such as sensitive domain and human oversight. This interface lowers the barrier to use. Rather than requiring a user to edit YAML, invoke scripts manually, or understand the internals of each agent, the dashboard presents governance evaluation as a guided form submission followed by structured results.

A final aspect of the opportunity concerns organizational learning. Each evaluation can potentially produce a repeatable set of gaps and controls that can be analyzed over time. Even in prototype form, the system illustrates how a governance lab could move from ad hoc discussion to a more disciplined review culture. Instead of debating every new AI project from scratch, teams could compare use cases against a common structure, notice recurring weaknesses, and progressively improve their policies, testing methods, and documentation standards. The project therefore addressed both an immediate operational problem and a longer-term capability-building opportunity.

4. Importance of the Project to the Organization

The importance of the UNICC AI Safety Lab project lies in its ability to create a practical bridge between AI innovation and AI governance. Organizations increasingly face pressure to adopt AI systems rapidly, yet poorly governed adoption can produce legal exposure, operational failures, reputational damage, and harm to affected individuals. A governance tool that improves pre-deployment review can therefore create substantial organizational value by reducing the likelihood of avoidable failure. In settings where AI is being proposed across multiple functional areas, the absence of such a process can lead to inconsistent standards and elevated risk.

For a governance-oriented organization, the project provides a way to translate abstract principles into operational review steps. The repository demonstrates this concretely.

The standards agent prompt is not merely descriptive; it instructs the model to identify triggered standards, explain alignment and gaps, and end with a deployment recommendation. The safety agent prompt requires scoring across six specified dimensions. The assurance agent prompt requires evaluation of evidence gaps, accountability structure, assurance controls, residual risk, and a recommendation. These structures make governance actionable. They move organizational review conversations from general concern to explicit findings and control recommendations.

The project is also important because it promotes explainability in governance operations. When stakeholders receive only a single pass or fail conclusion, trust in the review process may weaken. Developers may feel blocked by rules they do not understand, while risk leaders may be unable to show why approval was granted. By contrast, the UNICC AI Safety Lab prototype makes the intermediate reasoning visible. The dashboard exposes per-agent summaries, detailed lists of gaps and controls, and a chart of safety dimension scores. This helps align technical teams and governance teams around a shared evidence base.

An additional organizational benefit is modularity. Because agents are configured through YAML files and text prompts, the system can be extended without redesigning the whole platform. A sponsor could introduce a privacy specialist, sector-specific specialist, or jurisdiction-specific specialist in future iterations. That extensibility matters because AI governance requirements evolve. A useful governance platform must allow new review criteria to be added as regulations, internal policies, or organizational priorities change. The current project creates the architectural foundation for such growth.

Finally, the project has educational and demonstration value. Even if the current implementation is not production-grade, it is important because it helps stakeholders understand how multi-agent AI governance can work in practice. It demonstrates a sequence of software services, prompt engineering, aggregation logic, and user

interface design that can support AI review. This makes the project valuable both as a prototype for future organizational deployment and as a proof of concept that helps leadership, instructors, or sponsors visualize what an AI safety lab capability could look like.

5. Alternative Solutions Evaluated

Although the implemented solution follows a multi-agent architecture, several alternative solution patterns were plausible at the outset of the project. The first alternative would have been a fully manual governance checklist. In this approach, reviewers would fill out a structured document for each AI proposal and record findings against governance categories such as fairness, privacy, transparency, and oversight. The advantage of this method is simplicity and low technical overhead. However, it scales poorly, depends heavily on reviewer discipline, and provides limited automation for summarizing recurring issues across cases.

A second alternative would have been a single-agent or single-prompt architecture. Under that design, one large prompt would ask a language model to assess all relevant governance domains and produce one comprehensive response. This approach would reduce orchestration complexity and require fewer moving parts. However, it would blur distinct reasoning roles, make outputs harder to normalize, and reduce explainability. It would also complicate future extensibility because any changes to the

evaluation logic would affect one large, tightly coupled prompt rather than a set of specialized components.

A third alternative would have been a rules-only engine without generative reasoning. A purely rules-based system might rely on hard-coded questionnaires, weighted scoring rubrics, and deterministic logic. Such a design can be useful for consistency, especially when the review criteria are narrow and stable. Yet AI governance often requires interpretation of contextual details. For example, whether a data source is appropriate, whether current controls are sufficient, or whether residual risk is acceptable may depend on narrative context that cannot easily be reduced to a small number of rigid rules. A rules-only system therefore risks being too brittle for complex proposals.

A fourth alternative would have been direct use of a commercial external governance platform or a fully remote model API with embedded policy prompts. This might reduce local engineering work and potentially offer enterprise features more quickly.

Nevertheless, it introduces dependency, potential cost, privacy concerns, and reduced control over evaluation behavior. Because the project appears to emphasize local deployment using Ollama and llama3.2, the local-first alternative is better aligned with a safety lab concept where data handling, reproducibility, and technical transparency matter.

The implemented multi-agent local-first architecture was preferable because it balanced flexibility, explainability, and technical realism. It offers more structure than a single-

prompt model, more adaptability than a rules-only tool, and more automation than a manual checklist. At the same time, it remains small enough to be implemented as an academic prototype. This balance made it the most appropriate choice for the project scope.

6. Solution Evaluation Criteria and Selection Rationale

The selection of the final solution can be justified through a set of evaluation criteria derived from both governance needs and project constraints. The first criterion was explainability. Because AI governance decisions affect whether systems move forward, stakeholders must be able to understand the basis of a recommendation. The chosen multi-agent approach performs well on this criterion because each agent has a clearly defined role, prompt, and output schema.

The second criterion was modularity. The system needed to support change over time as governance frameworks evolve. YAML configuration files, prompt files, and separate services for each agent enable targeted updates without destabilizing the entire system. This makes the chosen architecture preferable to a monolithic prompt or tightly coupled codebase.

The third criterion was implementation feasibility within an applied project timeframe. A full enterprise governance platform with authentication, persistence, audit trails, and workflow routing would likely exceed the scope of a single applied project. By using

FastAPI, a React dashboard, shell scripts, and a local model endpoint, the project remained feasible while still demonstrating full-stack integration. The repository confirms that the system includes backend services, orchestration logic, user interface components, example payloads, and demo output, which is appropriate for a capstone-style prototype.

A fourth criterion was relevance to AI safety objectives. The project needed to do more than classify proposals. It needed to surface risks, gaps, and controls that map to real governance concerns. The chosen design explicitly includes triggered standards, safety dimension scores, evidence gaps, accountability structure, assurance controls, and aggregated cross-agent findings. This output is materially more useful for governance than a simple risk score.

A fifth criterion was privacy-conscious deployment. The choice to call a local Ollama endpoint rather than a remote API aligns with use cases that may involve sensitive organizational proposals. While the prototype still requires stronger operational safeguards for production, the architectural direction is consistent with the project's safety-lab purpose. On balance, the selected solution best satisfied the combined criteria of explainability, modularity, feasibility, governance relevance, and privacy-aware deployment, which explains the rationale for choosing the multi-agent architecture.

7. Approach and Methodology

The project followed an iterative applied-systems methodology that combined requirements interpretation, architecture design, prompt engineering, service implementation, orchestration logic, user interface development, and scenario-based testing. The first methodological stage was requirements framing. The repository contents indicate that the project was intended to evaluate AI use cases across standards, safety risks, and assurance and accountability. The README and architecture notes clarify that the system was built as a simulation of a real-world governance review process rather than a legal compliance engine. This framing informed all later design choices.

The second methodological stage was functional decomposition. Instead of treating AI governance as a single task, the project decomposed it into three specialized agents. This was not only an implementation convenience but a methodology choice. The architecture notes explain that separate agents create clearer role separation, easier debugging, and better explainability. Functional decomposition therefore served as both a software engineering strategy and a governance reasoning strategy.

The third stage was interface and schema design. The repository's schemas file defines an EvaluateRequest containing the use case title, description, intended users, affected groups, data sources, deployment context, current controls, and extra context. This

shared request object is important because it standardizes how all agents receive inputs. Each agent then returns a structured response that includes raw text and a parsed JSON payload. This structure supports reliable downstream aggregation and front-end rendering.

The fourth stage was prompt engineering for controlled JSON output. Each agent's prompt specifies its reasoning goals and output structure. The standards agent returns triggered standards, standards analysis, key gaps, required controls, and recommendation. The safety agent returns exactly six dimension scores, a list of high-priority risks, mitigations, and recommendation. The assurance agent returns evidence gaps, accountability structure, assurance controls, residual risk, and recommendation. Because language models can be inconsistent, the code also includes parsing and normalization logic that extracts JSON from raw output and repairs selected fields. The fifth stage was orchestration design. The orchestrator reads configuration from YAML, calls each agent's evaluate endpoint, normalizes results, builds a summary of cross-agent risks, gaps, and controls, and derives a final decision. This methodology reflects a controller pattern in which specialized analysis services feed a consolidating decision layer. The aggregation logic prioritizes rejection over flagging and flagging over conditional approval, which makes the final outcome conservative when serious concerns are present.

The sixth stage was front-end design for stakeholder usability. The React dashboard provides a form-based workflow, displays the current system decision, shows cross-agent findings, and visualizes safety scores with a bar chart. This made the prototype usable for demonstrations and helped verify that agent outputs could be interpreted by end users rather than only by developers.

The final stage was scenario-based testing using a sample payload for an AI hiring screening system. This scenario was appropriate because it is a recognized high-impact use case that activates multiple governance issues. The recorded demo output shows how the system produces an Approve with Conditions decision and identifies specific controls related to transparency, bias testing, data protection, human oversight, accountability, logging, and monitoring.

8. Project Objectives and Metrics for Project Evaluation

The project objectives can be expressed in measurable terms. The primary objective was to build a working prototype that accepts a structured AI use case description and returns a consolidated governance evaluation. Success for this objective required that the system run end to end, from user interface input to backend orchestration and final result display. The repository demonstrates this through the existence of launch scripts, FastAPI applications, a React front end, and a sample evaluation workflow.

A second objective was to implement specialized governance reasoning roles rather than a generic assessment. The evidence for success lies in the presence of three separate agents with distinct YAML configurations and distinct prompt files. Each agent produces different output sections. This indicates that role separation was achieved at the design and implementation level. A suitable metric for this objective is structural completeness: all intended agent types must exist, be configurable independently, and return schema-aligned outputs.

A third objective was to generate explainable outputs for governance review. The system was not meant to provide only a score; it was intended to produce interpretable risks, gaps, controls, and recommendations. Relevant metrics here include the presence of populated summary fields, the availability of per-agent reasoning, and the completeness of JSON parsing for all required sections. The demo output confirms that these explainable elements were produced.

A fourth objective was to support sensitive-domain scenario testing. The included sample payload for AI hiring screening represents such a scenario. A success metric for this objective is the system's ability to identify substantive governance concerns in a high-impact use case rather than defaulting to a trivial approval. The demo output's recommendation of Approve with Conditions, together with multiple identified gaps and controls, suggests that the prototype met this objective.

A fifth objective was usability for demonstration. The front end had to allow a non-programmatic submission flow and provide a readable result display. Metrics here include successful rendering of the input form, result blocks, and the bar chart of safety dimensions. Although this report did not execute the software locally, the repository contents indicate that these UI components were implemented.

A sixth objective was local-deployment feasibility. The environment example and scripts show that the project was designed to run against a local Ollama service with llama3.2.

A practical metric for this objective is the existence of reproducible local setup instructions and scripts. The README, .env example, requirements file, and run_all shell script collectively support this metric.

Overall, the project evaluation metrics were therefore a mix of technical completeness, role separation, output explainability, scenario relevance, usability, and local deployability. These metrics are suitable for an applied project because they evaluate whether the prototype solves the intended problem in practice rather than only in concept.

Objective	Representative Metric	Status
End-to-end prototype	Form input to final decision rendered	Achieved in repository design
Role specialization	Three distinct agents with	Achieved

	unique schemas	
Explainable outputs	Risks, gaps, controls, recommendations visible	Achieved
Sensitive-domain test case	Hiring-screening scenario evaluated	Achieved
Local deployment support	Ollama, requirements, run scripts documented	Achieved

9. System Architecture and Technical Design

The technical architecture of the UNICC AI Safety Lab prototype is a lightweight service-oriented design composed of specialist agent services, an orchestrator, and a browser-based front end. Each agent is implemented as a FastAPI application using a shared codebase that loads role-specific configuration and prompt files at runtime. This is an efficient design because common behavior such as request parsing, prompt construction, model calling, JSON extraction, and health endpoints can be reused while agent identity and purpose are varied through configuration.

At the core of each agent service is the evaluate endpoint. The agent application loads the YAML configuration selected through the AGENT_CONFIG environment variable. It

then loads the appropriate prompt text, builds a model prompt from the request payload, sends that prompt to the Ollama generate endpoint, attempts to parse JSON from the model's response, normalizes selected fields, and returns both the raw model text and the parsed output. This design offers transparency because the raw response is preserved while the parsed output supports downstream automation.

The shared schema object is an important part of the architecture. The EvaluateRequest model includes structured narrative fields that are broad enough to describe many AI use cases without becoming unmanageably complex. This model also includes an `extra_context` dictionary, which gives the system flexibility to pass contextual variables such as geography, whether the domain is sensitive, and whether human oversight exists. The use of a structured request model improves input consistency and allows the same payload to be evaluated by all agents.

The orchestrator functions as the system's decision engine. It reads a YAML file listing the agent endpoints, sends the payload to each agent, normalizes results, compiles cross-agent risks, gaps, and controls, and applies final decision rules. The decision logic is intentionally conservative. Any explicit reject leads to Reject. If there is no reject but there is a flag, the final decision is Flag for Review. If there is no reject or flag but at least one conditional recommendation, the final decision becomes Approve with Conditions. Only unanimous approval produces Approve. This aggregation design is simple, explainable, and appropriate for a risk-sensitive prototype.

The user interface is built with React, Vite, axios, lucide-react, and recharts. The dashboard includes a structured form, a decision panel, summary blocks, per-agent result cards, and a bar chart for safety dimension scores. The inclusion of a chart is significant because it helps users visually compare risk dimensions rather than reading only narrative text. The UI also includes toggles for sensitive domain and human oversight, which shows a deliberate effort to expose governance-relevant contextual settings to the user.

From an infrastructure standpoint, the system is intentionally minimal. Dependencies include FastAPI, uvicorn, requests, PyYAML, python-dotenv, pydantic, React, Recharts, and Vite. Startup is managed through shell scripts. This simplicity is advantageous for an applied project because it limits unnecessary complexity while still demonstrating full-stack integration. However, it also indicates areas for future enhancement, such as authentication, persistent storage, logging backends, versioned evaluations, and enterprise deployment controls.

Component	Primary Function
Standards Compliance Agent	Evaluates triggered standards, key gaps, controls, and recommendation
Safety Dimensions Agent	Scores six safety dimensions and proposes mitigations

Assurance & Accountability Agent	Assesses evidence, ownership, controls, and residual risk
Orchestrator	Calls agents, normalizes outputs, aggregates findings, derives final decision
React Dashboard	Captures input and displays summaries, charts, and per-agent outputs
Ollama / llama3.2	Provides local LLM inference for the agents

10. Detailed Component Analysis

The Standards Compliance Agent is responsible for external governance framework alignment. Its prompt references the EU AI Act, the White House Blueprint for an AI Bill of Rights, IEEE 7001, IEEE 7003, IEEE 7009, ISO/IEC 23894, and the UNESCO Recommendation on the Ethics of Artificial Intelligence. Its output schema requires an executive summary, triggered standards, standards analysis, key gaps, required controls, and a recommendation. This design is effective because it prevents the standards discussion from becoming a generic ethics narrative. Instead, it requires explicit

identification of which frameworks are implicated and what controls would strengthen readiness.

The Safety Dimensions Agent performs a bounded six-dimension review: harmfulness, bias and fairness, transparency, deception, privacy, and accountability. Its prompt explicitly requires six objects in the `dimension_scores` field, helping enforce consistency.

The application code further normalizes the output and computes high-priority risks from dimensions with scores above a threshold. This is a practical pattern because language-model outputs can be variable, and normalization helps maintain predictable downstream behavior.

The Assurance and Accountability Agent addresses implementation reality. It evaluates evidence quality, ownership, incident response, human oversight design, monitoring, logging, auditability, fallback procedures, and residual risk. This component is especially important because many discussions of AI safety stop at principles. By asking whether evidence exists, who owns decisions, and how incidents are escalated, this agent forces the review process to consider whether governance can actually be operationalized.

The aggregation layer deserves particular attention. There are effectively two pieces of aggregation logic present in the repository: a standalone `aggregate.py` module and the more fully integrated logic inside the orchestrator application. The orchestrator application builds cross-agent summaries and applies recommendation precedence, while the standalone aggregation module defines a decision order and deduplication

strategy. The coexistence of both suggests iterative development. It is common in applied projects for early utility functions to evolve into integrated application logic. This is acceptable, but in future versions the repository should consolidate and document the authoritative aggregation path more clearly.

The shell scripts are an important operational component. The `run_all` script starts three agent services and then the orchestrator. This script demonstrates that the system was intended to be runnable as a coordinated local environment rather than a set of isolated code samples. Although the script is simple, it gives the project practical credibility because it reflects how a reviewer would actually launch the prototype. In applied work, such operational details matter because prototypes that cannot be started reliably are difficult to demonstrate or validate.

The example payload and demo output serve as test artifacts. The sample payload describes an AI hiring screening system using resume text, application history, and screening questions before recruiter review. The demo output then shows a consolidated Approve with Conditions decision, along with specific risks, evidence gaps, and controls. These artifacts strengthen the project because they provide a traceable example of system behavior.

11. Demonstration Scenario and Results Interpretation

The demonstration scenario included in the repository concerns an AI hiring screening system. This is a strategically strong example because hiring is a sensitive domain that can materially affect individuals' opportunities and rights. The sample use case states that the model ranks job applicants using resume text, application history, and screening questions before human recruiter review. The extra context also identifies the domain as sensitive and indicates that human oversight exists. These inputs create a realistic governance case rather than a trivial or low-risk scenario.

The recorded output shows that the final system decision was Approve with Conditions. This result is significant. It indicates that the prototype did not respond with blanket rejection merely because the domain is high impact, nor did it issue approval based on superficial existing controls. Instead, it produced a conditional outcome tied to specific gaps and mitigations. That is precisely the kind of balanced result a governance tool should aim for. The purpose of governance review is not to stop all innovation but to separate ready proposals from proposals that require additional controls before deployment.

The cross-agent summary identified risks such as harmfulness, privacy, and accountability. It also identified gaps including the need for human oversight guidelines, stronger security measures, diversity and inclusion in development, insufficient bias

performance evidence, lack of transparency into algorithmic logic, and lack of clear documentation about training data sources. These findings are appropriate for an AI hiring use case and show that the agents are focusing on relevant governance concerns rather than generating generic filler text.

The controls recommended in the demo output also align with established governance practice. They include transparent and explainable decision-making processes, robust security measures, regular testing for bias and fairness, anonymization and secure storage of applicant data, a clear accountability framework, a data audit log, regular validation on diverse datasets, an incident response plan, a human oversight design, and logging and monitoring capabilities. The breadth of these controls shows the advantage of combining standards, safety, and assurance viewpoints.

The safety dimension scores in the demo are also revealing. Privacy received the highest score, while harmfulness and accountability were also elevated. Bias and fairness, transparency, and deception scored lower but remained present in the analysis. In a prototype system, the exact numbers are less important than the structured discipline of requiring each dimension to be assessed. This helps reviewers compare where attention is most needed and prevents important dimensions from being omitted simply because they are not top of mind during a manual review.

Overall, the demonstration results support the conclusion that the prototype achieved its intended purpose. It can ingest a realistic sensitive-domain use case, generate

specialized analyses, aggregate the results, and present them in a usable interface. The output is not perfect or legally authoritative, but it is substantive enough to support governance discussion and next-step planning.

Finding Type	Illustrative Result from Demo
Final decision	Approve with Conditions
Top cross-agent risks	Harmfulness, Privacy, Accountability
Key standards concerns	Oversight, transparency, security, fairness testing
Assurance concerns	Evidence gaps, accountability ownership, incident response
Illustrative controls	Audit logs, diverse validation, oversight design, monitoring

12. Risk Analysis

Risk analysis was central to both the project subject matter and the project delivery process. The first category of risk concerns governance accuracy. Because the system uses a language model to interpret narrative descriptions and generate structured findings, there is a risk of hallucinated controls, omitted concerns, unstable outputs, or

overconfident recommendations. The project mitigated this risk in part by constraining each agent through structured prompts and JSON schemas and by framing the standards agent as an alignment evaluator rather than a legal compliance judge. Even so, model output quality remains a material risk in any production deployment.

A second category of risk concerns input incompleteness. Governance conclusions are only as reliable as the information provided in the use case submission. If a user omits material facts about data sources, oversight, deployment scope, or controls, the evaluation may be misleading. The structured request form mitigates this somewhat by prompting for key categories, but it does not eliminate the problem. Future versions would benefit from validation rules, minimum evidence requirements, document attachments, and workflow states indicating that a case is incomplete.

A third category of risk concerns privacy and information handling. Even though the design uses a local Ollama endpoint, prompts may still include sensitive organizational information. Without authentication, encryption, role-based access control, logging governance, and data retention policies, such a system could become a new source of information risk. The current prototype is adequate for demonstration, but a production version would need significantly stronger controls.

A fourth risk concerns governance drift. AI regulations and standards change rapidly. A system that references governance frameworks through static prompt files can become outdated if not actively maintained. This is especially relevant for areas such as the EU

AI Act implementation timeline and related guidance. The multi-agent modular design helps because prompts can be updated independently, but maintenance discipline is still required.

A fifth risk concerns user over-reliance. There is a danger that stakeholders may treat the system's recommendation as definitive rather than advisory. This is particularly problematic in high-stakes domains. The project mitigates this partially by presenting outputs as recommendations accompanied by gaps and controls. Nevertheless, the user interface and documentation should continue to stress that the tool supports human review and does not replace legal, compliance, or executive judgment.

A sixth risk concerns engineering robustness. The local shell-script startup approach is sufficient for a prototype but may be fragile in multi-user environments. Error handling, service discovery, structured logging, queueing, retries, and deployment packaging would require improvement for operational adoption. These technical risks do not diminish the project's prototype value, but they should be recognized in any plan for future transition.

13. Issues Encountered and How They Were Handled

Although the repository does not include a detailed engineering issue log, the codebase itself reveals several categories of issues that were likely encountered during development. One issue is the inconsistency of language-model JSON output. The

presence of helper functions that strip code fences, locate JSON substrings, attempt reparsing, and normalize output strongly suggests that raw responses were not always immediately machine-readable. This is a common issue in applied LLM systems. The project addressed it by adding parsing safeguards and normalization logic rather than relying on perfect model behavior.

Another issue is schema variability across agents. The safety agent, for example, produces dimension scores that may vary in content structure, and the code normalizes these to a consistent list. Likewise, the orchestrator contains helper functions that convert strings, lists, and dictionaries into normalized string lists. This indicates that the developer encountered output heterogeneity and handled it by creating post-processing functions that improve consistency before aggregation and display.

A third issue concerns the balance between prototype simplicity and governance completeness. The system includes useful functionality but does not yet include persistence, authentication, or workflow routing. This suggests that scope management was necessary. Rather than attempting to build every enterprise feature, the project focused on the core governance evaluation loop: input, specialist analysis, aggregation, and presentation. This was an appropriate handling strategy for an applied project because it preserved a demonstrable end-to-end result.

A fourth issue likely involved front-end and back-end integration. The dashboard posts directly to the orchestrator endpoint and then expects a structured result object with

summary and agent_results sections. The fact that the UI includes per-agent cards and a safety dimension chart indicates that response structures had to be stable enough for rendering. The normalization work in the backend likely helped resolve front-end rendering problems by producing predictable shapes.

A fifth issue concerns prompt specificity. The safety prompt in particular is strict about returning exactly six dimension objects. Such instruction usually emerges from earlier iterations in which models either omitted dimensions or added extra explanatory material. By tightening the prompt and combining it with application-level normalization, the project handled this issue pragmatically.

Overall, the project handled its issues in a mature prototype-oriented way. Instead of abandoning functionality because model outputs were imperfect, it added control layers around the model. Instead of overscoping the platform, it prioritized an end-to-end demonstration. These were sound choices for an applied project.

14. Detailed Project Chronology and Critique

The project chronology can be reconstructed from the architecture and repository structure. The initial stage likely involved defining the problem and selecting the AI governance domain as the project focus. At this stage, the concept of an AI safety lab and governance evaluation system would have been established, together with the decision to use a high-impact scenario such as AI hiring for demonstration purposes.

The next stage was architectural decomposition. The project notes explain why three agents were chosen and why a single giant agent was not preferred. This indicates that design-level reasoning preceded implementation. The decision to separate standards, safety, and assurance into distinct roles shaped the rest of the work and was a strong early choice because it gave the project a clear structure.

The following stage was backend implementation. Shared schemas, the reusable agent application, model-calling logic, and configuration loading were developed. This stage likely included several rounds of prompt revision, output parsing fixes, and YAML refinement. The presence of normalization functions and parsing safeguards suggests iterative testing and correction during development.

After agent development, the orchestrator was implemented to call each service, combine outputs, and derive a final decision. This likely required work on endpoint coordination, error handling, and summary logic. Once the orchestrator was functioning, the project moved into front-end development so that the prototype could be demonstrated through a browser-based dashboard rather than command-line inputs alone.

The later stage involved preparing scripts, example payloads, demo output, and documentation. The README consolidates setup steps, folder structure, example use cases, and architecture summaries. The example payload and demo output likely served as validation artifacts for demonstrating that the system could run end to end.

A critique of the chronology suggests two areas that could have been improved. First, more visible documentation of test cases, limitations, and evaluation criteria inside the repository would strengthen project traceability. Second, the repository would benefit from clearer differentiation between prototype assumptions and production requirements. Even so, the chronology appears coherent: concept definition, architecture selection, backend implementation, orchestration, front-end integration, and demonstration packaging. That is a credible progression for an applied technical project.

15. Lessons Learned

The project produced lessons in AI governance, software architecture, and project management. One major lesson is that AI governance becomes more actionable when it is decomposed into distinct reasoning roles. In theory, concepts such as fairness, transparency, and accountability are often discussed together. In practice, separating them into specialist services improves both clarity and implementation discipline. The project shows that modular governance reasoning is not just conceptually elegant but operationally useful.

A second lesson is that prompt engineering alone is insufficient for reliable system behavior. The code demonstrates that application-level safeguards are necessary. JSON parsing helpers, normalization functions, and explicit schema design all play critical roles

in turning model output into a usable service. This is an important practical lesson for any LLM-based system: engineering reliability comes from the entire pipeline, not just from the prompt.

A third lesson is that explainability matters at multiple levels. It is not enough for the final system decision to be explainable. The architecture itself should also be explainable. By making the roles of the standards agent, safety agent, and assurance agent explicit, the project improves stakeholder trust in the review process. Users can understand not only what the answer is but how the system was organized to produce it.

A fourth lesson is that prototype scope must be controlled. The project avoided trying to solve every enterprise challenge at once. It did not attempt a full governance workflow platform with user management, document storage, audit records, and integration into institutional approval systems. Instead, it demonstrated the most important loop: capture a use case, evaluate it from multiple governance perspectives, aggregate the findings, and present the result. This scope discipline increased the likelihood of producing a complete and demonstrable artifact.

A fifth lesson concerns the value of local deployment for sensitive use cases. The choice to use Ollama and llama3.2 supports the broader principle that governance tools should minimize unnecessary data exposure. Even though local hosting is not a complete privacy solution, it reflects good architectural instincts for safety-related applications.

Finally, the project reinforced the lesson that AI safety is not only about preventing harmful model outputs. It is also about decision process quality, documentation, control sufficiency, ownership, monitoring, and escalation. The system's assurance component makes this lesson visible and shows why governance capability must extend beyond model evaluation alone.

16. Conclusions and Summary

The UNICC AI Safety Lab applied project successfully produced a working prototype of a multi-agent AI governance evaluation system. The repository materials show that the system includes specialist agent services, an orchestrator, a browser-based dashboard, configuration files, launch scripts, and demonstration artifacts. The project addressed a real and growing need: organizations require structured, explainable methods for reviewing AI use cases before deployment, especially in sensitive and high-impact domains.

The most important contribution of the project is architectural. By separating standards, safety, and assurance into distinct agents and then aggregating their outputs, the system creates a governance workflow that is more interpretable and extensible than a monolithic alternative. This architecture also aligns well with the complexity of AI governance, where legal alignment, ethical risk, operational readiness, and accountability cannot be responsibly collapsed into a single undifferentiated question.

The project also demonstrated practical software integration. The FastAPI services, Ollama model interface, YAML-driven configuration, and React front end combine into a coherent full-stack prototype. The included AI hiring scenario illustrates that the system can generate substantive governance outputs, identify relevant controls, and avoid simplistic approvals. For an applied project, this is a meaningful result because it proves not only that the concept is well reasoned but that it has been implemented.

At the same time, the project should be understood as a prototype rather than a final operational platform. It does not yet include all of the controls, governance processes, and enterprise features necessary for institutional deployment. Nevertheless, the prototype is strong enough to support demonstrations, structured sponsor discussion, and future enhancement planning. In that sense, it achieved the core objective of the applied project: to deliver a concrete technical artifact that addresses a real organizational need and provides a credible foundation for future work.

In summary, the UNICC AI Safety Lab project is an effective applied demonstration of how modular AI governance review can be designed and implemented. It translates AI safety concerns into a usable evaluation workflow and provides an architectural foundation for more advanced governance tooling.

17. Limitations, Recommendations, and Scope for Future Work

The prototype has several important limitations. The first is that it depends on prompt-driven LLM reasoning without a formal evaluation benchmark. The project includes a demonstration case, but it does not yet provide systematic validation across a broader set of use cases. Future work should therefore include a curated evaluation dataset spanning different domains, risk levels, and governance patterns so that output quality can be assessed more rigorously.

A second limitation is the absence of persistent storage and audit history. Governance review is often valuable not only at the moment of decision but also months later, when stakeholders need to understand what information was considered, what controls were requested, and why a deployment decision was made. Future versions should include storage for submissions, responses, timestamps, reviewer notes, and version history.

A third limitation is that the system currently appears to rely on synchronous calls and simple shell-based startup. For a demonstration this is sufficient, but an operational version would require more robust deployment patterns, structured logging, retry strategies, service monitoring, containerization, and possibly asynchronous task execution.

A fourth limitation is the absence of user roles, authentication, and access control. A governance platform may need different permissions for submitters, reviewers, policy

leads, auditors, and administrators. Security design should therefore be a priority in any future deployment roadmap.

A fifth limitation is standards maintenance. The standards prompt references important frameworks, but governance frameworks evolve and organizational policy interpretations differ. Future work should add versioned policy packs, update workflows, and possibly retrieval support from maintained policy documents so that evaluations remain current.

A sixth limitation is that the prototype focuses on pre-deployment review. AI governance also requires post-deployment monitoring, incident response, periodic re-evaluation, drift analysis, and control verification. Future work should therefore connect the assessment tool to a broader lifecycle governance model.

Recommended future enhancements include: a case management database; document upload and evidence tracking; reviewer workflows; benchmarking across sectors; configurable scoring policies; exportable reports; multilingual support; stronger explainability summaries; and connectors to organizational policies and risk registers.

The scope of this applied project was appropriately limited to the prototype core, but future work can build directly on the architecture already created.

18. Literature Survey

A sound literature and practice foundation for this project comes from the convergence of regulatory, ethical, and risk-management frameworks. The European Union's AI Act establishes a risk-based framework for AI and specifically highlights the governance importance of high-risk uses, human oversight, and transparency in decision processes. The European Commission's implementation materials also emphasize that the AI Act is intended to foster trustworthy AI while differentiating obligations by risk category. This is directly relevant to the project because the prototype's standards agent and sensitive-domain example use case mirror the need for structured risk-aware review.

The White House Blueprint for an AI Bill of Rights provides another important practice lens. Its emphasis on safe and effective systems, algorithmic discrimination protections, data privacy, notice and explanation, and human alternatives reinforces the dimensions explored in the project. Even though the Blueprint is not a binding universal standard, it remains an influential articulation of governance principles for automated systems affecting people. The project's safety and assurance outputs align with these themes by surfacing bias, privacy, transparency, and oversight concerns.

NIST's AI Risk Management Framework is also highly relevant because it translates trustworthy AI aspirations into risk-management practice. NIST states that the framework is intended for voluntary use and to improve how trustworthiness

considerations are incorporated into design, development, use, and evaluation. The UNICC AI Safety Lab prototype fits well within that perspective because it functions as a pre-deployment decision-support mechanism that helps frame and assess risk rather than as a narrow compliance checklist.

UNESCO's Recommendation on the Ethics of Artificial Intelligence contributes a global and human-rights-centered perspective. UNESCO emphasizes that the protection of human rights and dignity is central, while also highlighting transparency, fairness, inclusiveness, and human oversight. This is especially important for a governance lab concept because governance cannot be reduced to legal minimalism alone. The project's multi-agent structure reflects this broader view by combining standards alignment with ethical-safety review and operational assurance.

Relevant standards and engineering guidance also come from IEEE and ISO materials cited directly in the repository prompts, including IEEE 7001, IEEE 7003, IEEE 7009, and ISO/IEC 23894. While the prototype does not embed full text from these standards, their inclusion is meaningful because it signals a design orientation toward transparency, bias considerations, fail-safe design, and AI risk management. In practice, the project operationalizes these themes through required controls, evidence gaps, and accountability recommendations.

The literature and standards reviewed suggest that best practice in AI governance has several recurring elements: a risk-based approach, explicit human oversight,

documentation and transparency, bias and fairness testing, privacy and security controls, defined accountability, and ongoing monitoring. The prototype incorporates each of these elements to some degree. Its greatest literature-aligned strength is not exhaustive legal precision but structural fidelity to the core themes that responsible AI governance frameworks repeatedly emphasize.

19. Situational Analysis

The situational environment for the UNICC AI Safety Lab project is shaped by two opposing forces. On one side, organizations face strong incentives to deploy AI quickly to improve efficiency, automate classification and decision support, reduce costs, and remain competitive. On the other side, AI deployment is increasingly subject to scrutiny regarding fairness, privacy, transparency, accountability, and compliance with emerging regulations. This creates a classic governance tension: innovation pressure increases the demand for speed, while safety and policy pressure increase the demand for deliberation and documentation.

A SWOT perspective clarifies the situation. The project's strengths include modular architecture, explainable outputs, local-model support, and a working end-to-end prototype. Its weaknesses include lack of persistence, limited enterprise controls, and dependence on prompt quality. Opportunities include expansion into additional governance domains, integration into institutional review processes, and use as a

repeatable teaching or evaluation platform. Threats include regulatory change, model-output instability, stakeholder over-reliance, and operational security challenges if the platform were deployed without stronger controls.

Porter's Five Forces can also be interpreted in this context. Competitive rivalry is rising because many vendors and internal teams are building AI governance tools. Buyer power can be high because organizations have options ranging from manual review to vendor platforms to internally built systems. Supplier power is mixed; local model hosting reduces dependency on external model providers, but the system still depends on the quality and suitability of the chosen model. The threat of substitutes is significant because governance can be performed manually or through generic risk frameworks. The threat of new entrants is also meaningful because lightweight LLM-enabled review tools can be assembled relatively quickly. These forces imply that differentiation must come from explainability, integration quality, and trustworthiness rather than novelty alone.

The situational analysis therefore supports the relevance of the project. A lightweight but well-structured governance review prototype can help organizations respond to this environment by creating more disciplined decision processes without requiring immediate investment in a large enterprise platform. The project is well positioned as a transitional capability: more rigorous than informal review, yet more feasible than a fully industrialized governance suite.

20. End-to-End Workflow Analysis

The end-to-end workflow begins when a reviewer or project owner enters a proposed AI use case through the dashboard. At that point, the workflow already creates governance discipline because the submitter must articulate the use case title, intended purpose, affected groups, data sources, deployment context, and existing controls. In many organizations these items are scattered across slide decks, emails, and conversations. Requiring them in a single structured submission increases clarity before any model analysis occurs.

Once the payload is submitted, the orchestrator becomes the central workflow manager. It does not itself perform all substantive reasoning. Instead, it routes the same request to each specialist agent so that each can interpret the case using its assigned perspective. This routing step is methodologically important because it ensures that all agents operate on a common factual basis. Without that consistency, different reviewers might unknowingly analyze different versions of the same proposal and generate misleading disagreement.

Each agent then transforms the structured request into a role-specific prompt. The standards agent emphasizes external governance frameworks, the safety agent emphasizes harm dimensions, and the assurance agent emphasizes implementation and accountability. This means the system's workflow is not just technically sequential; it is

cognitively diversified. The design intentionally creates multiple structured viewpoints on the same problem rather than a single generic answer.

After each agent returns its result, the orchestrator performs normalization and aggregation. This stage is where prototype-level reliability is most visible. Raw model output is often inconsistent in formatting, terminology, or list shape. The orchestrator and shared agent code reduce these inconsistencies so that a summary can be built. Governance tooling depends heavily on this stage because without normalization the user interface would present fragmented or incomparable findings.

Finally, the dashboard presents the results as a governance package rather than an opaque score. The user can review the system decision, the cross-agent summaries, the safety chart, and the detailed agent cards. This workflow demonstrates a critical principle of AI governance systems: useful outputs do not end at a recommendation. They must provide enough structure for stakeholders to act on the recommendation, discuss it, and challenge it where necessary.

21. Data Flow and Information Handling

Although the project does not implement a database, its information flow can still be analyzed carefully. Input data begins as a structured JSON request, not as a collection of ad hoc text fragments. This matters because governance analysis depends on preserving the semantic roles of information. Intended users, affected groups, deployment context,

and controls are not interchangeable categories; they shape different governance conclusions. The shared schema preserves that distinction.

The next important information handling step is prompt construction. The agent application concatenates the system prompt with user-provided context fields. This design is efficient but also highlights why prompt-layer privacy discipline matters. If future users submit highly sensitive organizational information, prompt construction becomes a de facto sensitive-data handling operation. The local-model design reduces external transmission risk, but internal access controls would still be required in a mature deployment.

The model response is handled in two forms: raw text and parsed JSON. Preserving the raw text is a sound decision for transparency and debugging because it allows developers and reviewers to see what the model actually produced before normalization. Parsed JSON, meanwhile, is necessary for the orchestrator and front end. Keeping both forms provides a balance between machine usability and reviewability. The current prototype appears to keep data in transit and memory during execution rather than in persistent storage. This simplifies the architecture but limits auditability and reproducibility. In a mature governance lab, the information lifecycle would need clear rules for retention, deletion, access, and export. These controls would be especially important if the platform handled vendor proposals, internal strategy documents, or regulated personal data.

From an AI safety perspective, the project's data flow illustrates an important governance lesson: information architecture is itself part of governance. A tool that evaluates privacy, oversight, and accountability must also be designed so that its own handling of user-provided governance data is disciplined, secure, and auditable.

22. Governance Mapping to Recognized Framework Themes

The project's standards agent aligns strongly with the broader theme of risk-based AI governance. The European Commission's AI Act materials explain that the Act differentiates obligations by risk and emphasizes trustworthy AI, human-centric safeguards, and protection of fundamental rights. The project reflects that logic by treating hiring as a sensitive use case and by looking for oversight, transparency, and safeguards rather than assuming that model performance alone determines readiness. The White House Blueprint for an AI Bill of Rights highlights safe and effective systems, protection against algorithmic discrimination, data privacy, notice and explanation, and human alternatives or fallback. These themes map well to the project's safety and assurance layers. The safety agent addresses discrimination, privacy, and transparency directly, while the assurance agent addresses human oversight, escalation, and fallback-oriented control design. In other words, the project operationalizes policy themes through architecture rather than leaving them as abstract principles.

NIST's AI Risk Management Framework is centered on the practical incorporation of trustworthiness considerations into design, development, use, and evaluation. The project mirrors this orientation by functioning as a pre-deployment review support mechanism. It is not purely a post hoc audit and not purely a policy document; it is a tool intended to shape earlier-stage decision making about whether a use case is sufficiently governed to move forward.

UNESCO's human-rights-centered approach is also visible in the prototype. UNESCO emphasizes human dignity, fairness, inclusion, and oversight. The project's requirement to specify affected groups and current controls, together with the safety and assurance agents' focus on harm, accountability, and reviewability, moves the evaluation toward these concerns. This is important because governance tools can otherwise drift into narrow compliance logic while missing broader societal risk.

This mapping exercise shows that the project does not merely mention well-known governance frameworks. It embodies recurring themes from those frameworks in its technical design. That design choice strengthens the project's credibility as an applied governance prototype.

23. Testing Strategy and Validation Considerations

The repository includes evidence of scenario-based testing through the sample payload and demo output. This indicates that at least one end-to-end validation path was

established. For an applied project prototype, this is a necessary baseline because it confirms that the system components can communicate and that the front end can present the final structure. However, scenario-based success does not by itself constitute robust validation.

A more mature validation strategy would include a test suite of use cases spanning multiple sectors and governance profiles. For example, a low-risk internal productivity assistant, a customer support chatbot, a recommendation system, a healthcare triage assistant, and a credit scoring use case would test whether the system differentiates risk and controls appropriately. Such a suite would make it possible to compare expected versus actual recommendations and to identify where prompts or normalization logic need refinement.

Validation should also include schema compliance testing. Because the system relies on JSON structures for aggregation and display, automated checks should confirm that each agent returns all required sections, that fields expected to be lists are lists, and that dimensions or recommendations conform to allowable values. The code already performs partial normalization, which indicates awareness of this need. The next step would be formal automated tests around those behaviors.

Human evaluation would also be valuable. Governance experts, technical reviewers, and non-specialist users could review the same cases and compare their judgments with the system outputs. The purpose would not be to force full agreement but to identify

whether the system consistently surfaces the right issues for discussion. In governance contexts, surfacing the right questions can be as important as predicting a final recommendation.

Therefore, while the prototype demonstrates functional validation, a future version should adopt layered validation: functional integration tests, schema conformance tests, scenario benchmarking, and expert comparative review. This would strengthen confidence that the system is not only runnable but also substantively dependable.

24. Ethical Critique of the Prototype

An ethical critique of the prototype must acknowledge both its strengths and its tensions. Its strongest ethical contribution is that it resists the reduction of AI review to a binary compliance checkbox. By requiring discussion of harms, privacy, transparency, accountability, evidence quality, and human oversight, the system encourages richer review than many ad hoc processes. Ethically, that is a meaningful advance because it makes the governance process more reflective and less superficial.

At the same time, the prototype raises a meta-governance question: to what extent should an AI system be used to evaluate other AI systems? This is not a contradiction, but it does create a responsibility for humility and control. If the reviewing system becomes too authoritative, organizations may outsource critical judgment to an automation layer that itself is imperfect. The project partly addresses this by presenting

recommendations and supporting rationale rather than a hidden score, yet this risk remains important.

Another ethical issue concerns the possibility of encoded policy bias. Prompt files inevitably reflect assumptions about what counts as good governance. Those assumptions may be reasonable, but they are not neutral. Future versions of the platform should therefore make prompt and policy choices more visible, versioned, and reviewable so that stakeholders can understand the normative framework built into the tool.

There is also the question of accessibility and inclusion. A governance platform should not require advanced technical fluency to use effectively. The dashboard helps with this, but future refinement should consider language clarity, multilingual presentation, accessibility features, and explanatory help text for less technical users. Governance is weakened when only specialists can meaningfully participate.

The ethical conclusion is therefore balanced. The prototype is ethically constructive because it improves the structure and transparency of AI review. However, it must itself be governed carefully if it evolves into an institutional tool. Good governance systems do not exempt themselves from governance; they make their own assumptions, limits, and control needs explicit.

25. Managerial Implications

For managers, the most important implication of the project is that AI governance can be operationalized through workflow and system design rather than treated only as policy language. Executives often ask how governance can be embedded in project decisions without stopping innovation. The prototype offers one answer: structure the review around required inputs, specialist perspectives, explainable outputs, and actionable controls. This turns governance into a repeatable management process.

A second managerial implication concerns accountability. Many organizations struggle because AI initiatives move forward without clear ownership for exceptions, incidents, monitoring, or post-deployment review. The project's assurance component shows that accountability can be treated as a first-class review question. Managers can use this insight to improve governance charters, committee roles, and approval pathways even apart from the prototype itself.

A third implication is organizational learning. If future versions of the system stored results over time, management could analyze patterns in rejected or conditionally approved use cases. This would help identify recurring weaknesses such as poor documentation, weak bias evidence, or insufficient privacy controls. Such trend analysis could guide training investments, policy revision, and capability-building priorities.

A fourth implication is cross-functional alignment. AI review often fails because legal, technical, product, and operations teams speak different languages. A multi-agent governance interface can serve as a translation layer by organizing concerns into categories that all stakeholders can see. Managers should recognize that one value of such a tool is not merely decision automation but better structured conversation across functions.

Finally, the project suggests that governance capability should be built incrementally. Managers do not need to begin with a fully mature enterprise platform. A lightweight but well-structured prototype can still create value by standardizing inputs, revealing control gaps, and improving documentation quality. From a managerial perspective, this makes the project a useful model for phased governance capability development.

26. Deployment Roadmap for a Future Version

A realistic deployment roadmap would begin with hardening the current prototype rather than replacing it. Phase one would focus on engineering stabilization: containerization, environment management, structured logging, authentication, and a persistent datastore. These are foundational steps because they transform the prototype from a demo environment into a controlled application service.

Phase two would focus on governance workflow maturity. This phase would add case submission states, reviewer assignment, comment and revision cycles, evidence

attachments, exportable reports, and approval history. The goal would be to support not just one-time evaluation but a multi-step organizational review process. This is where the AI safety lab concept would become a practical governance service rather than only a technical demonstration.

Phase three would focus on quality assurance and policy maintenance. It would include benchmark scenarios, prompt versioning, policy pack management, and scheduled framework updates. Because governance requirements change, maintainability must be treated as an explicit product feature rather than an afterthought.

Phase four would expand analytical depth. Additional agents could be added for privacy law, sector-specific compliance, model documentation review, red-team scenario analysis, or post-deployment monitoring readiness. The present architecture is well suited to such expansion because its agent-oriented design already assumes modular growth.

Phase five would connect the platform to organizational systems such as risk registers, document repositories, ticketing systems, or model inventory tools. At that point, the platform would support an AI governance lifecycle rather than a standalone review task. The roadmap therefore shows that the current project is a credible base layer from which a much more capable governance environment could be developed over time.

Appendices

Appendix A. Project Acceptance Document Placeholder



Sponsor's Project Acceptance Document – PART 1

This document is the means by which your project sponsor formally agrees that your project has been satisfactorily completed and that it meets the project goal and objectives that were set at the onset of the project. It is therefore important that you describe the goal, objectives, and related metrics in the appropriate section below. The "PLAN" section is to be prepared at the beginning of the project and the "RESULTS" and "ACCEPTANCE" sections after your project has been completed. Your sponsor should provide input and sign where indicated. The signed document will also be a required section in your Project Final Report. This document is a template whose sections may be expanded as necessary.

Project Title: UNICC AI Safety Lab: Multi-Agent AI Safety Evaluation and Decision Support Platform
Student Name: Tongye Ding
Sponsoring Organization: UNICC
Project Sponsor Name and Title: Andrew Fortino
Project Sponsor Contact Information (email and phone): agf249@nyu.edu

PROJECT PLAN

At project start, show the project goal; the project objectives and related metrics to be used to show successful project completion. Sponsor should sign to indicate agreement.

✓ **Project Goal** AF
Design and develop a functional AI safety lab prototype that evaluates AI-generated risks using a multi-agent expert framework and provides explainable safety assessments to support responsible AI governance.

✓ **Objective #1** AF Develop a working prototype including orchestrator, expert modules, risk scoring logic, and user interface. Success metric: core system components function as designed.

✓ **Objective #2** AF Validate the platform through use cases demonstrating detection, routing, and safety evaluation performance. Success metric: successful completion of representative test scenarios.

✓ **Objective #3** AF Assess the platform's effectiveness for supporting explainable and structured AI safety decision-making. Success metric: documented evaluation results and stakeholder feedback.

✓ **Objective #4:** Develop feasibility and implementation recommendations covering scalability, governance, and enterprise application potential. Success metric: completed strategic recommendations. AF

✓ **Objective #5:** Complete all capstone deliverables including documentation, final report, and presentation. Success metric: all deliverables submitted on schedule. AF

07-Part1 SponsorsProjectAcceptanceDocument V3.3.docx

I agree with the above planned project goal, project objectives, and related metrics.

Andres Fortino

Project Sponsor Signature

Date:

Printed Name:

Andres Fortino
Please print in English

Sponsor's Project Acceptance Document -Part 2

PROJECT RESULTS

STUDENT: Include below the project goal and ALL the objectives in your project, which you detailed in Project Plan section (07-Part1). You may add additional objectives, or add space, as needed.

Planned Start Date: January 2026

Planned End Date: May 2026

Actual Start Date: January 2026

Actual End Date: May 2026

If actuals differ from planned dates, the revised dates (Actual) are accepted by the sponsor if initialed here: **Sponsor Initials** _____

Project Goal Was the project goal achieved as planned? ☒ Yes ☐ No, Reason missed: _____
If NO, please explain why this is an acceptable deviation. _____ **Sponsor Initials** *AF*

Project Objective #1: <as shown above in Plan section (Part1)>
Did the student's project meet this objective with associated measures and metrics as established at project inception? **Objective#1** ☒ has or ☐ has not been met. **Sponsor Initials** *AF*
If not met please explain why this is or is not an acceptable deviation.

Project Objective #2: <as shown above in Plan section (Part1)>
Did the student's project meet this objective with associated measures and metrics as established at project inception? **Objective#2** ☒ has or ☐ has not been met. **Sponsor Initials** *AF*
If not met please explain why this is or is not an acceptable deviation.

Project Objective #3: <as shown above in Plan section (Part1)>
Did the student's project meet this objective with associated measures and metrics as established at project inception? **Objective#3** ☒ has or ☐ has not been met. **Sponsor Initials** *AF*
If not met please explain why this is or is not an acceptable deviation.

PROJECT ACCEPTANCE

Sponsor: Please show your evaluation of student's work, add comments, sign below

SPONSOR: Sponsor's evaluation of student's performance on this project:
Select One ☒ A ☐ A- ☐ B+ ☐ B ☐ B- ☐ C+ ☐ C ☐ C- ☐ F

Sponsor's Comments: _____

SPONSOR: Select One ☒ Project was completed satisfactorily and is hereby accepted.
☐ Project was completed satisfactorily but did not meet all objectives, as shown above. Nevertheless, the Project is accepted.
☐ Project was NOT completed satisfactorily and is NOT accepted.
Please explain why not accepted in Sponsor's Comments section, above.

Andres Fortino

Project Sponsor Signature

Date:

Tongye Ding

Student Signature

Date:

April 25, 2026

07-Part2 SponsorsProjectAcceptanceDocument V3.3.docx

All signatures in English. Sign by hand or use e-signature. Do not type or use handwriting font.

Appendix B. Project Plan, Baseline, and Actual WBS

A practical baseline work breakdown for the project would include: problem framing; governance framework selection; architecture design; prompt design; backend agent implementation; orchestration implementation; front-end dashboard development; scenario testing; documentation; and final report preparation. Actual execution likely involved several iterations between prompt design, backend normalization logic, and front-end integration as issues with JSON structure and display were addressed.

The principal deviation from an ideal baseline plan was likely the amount of time required for iterative prompt control and output normalization. LLM-based systems frequently consume more implementation effort in validation and guardrail design than initially expected. This deviation is common and does not represent poor planning; rather, it reflects the practical reality of turning generative output into reliable structured application behavior.

Appendix C. References and Bibliography

European Commission. (2026). AI Act. Shaping Europe’s digital future. <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>

NIST. (2023). AI Risk Management Framework (AI RMF 1.0). <https://www.nist.gov/itl/ai-risk-management-framework>

UNESCO. (2024). Recommendation on the Ethics of Artificial Intelligence.

<https://www.unesco.org/en/artificial-intelligence/recommendation-ethics>

White House Office of Science and Technology Policy. (2022). Blueprint for an AI Bill of Rights. <https://www.govinfo.gov/app/details/GOVPUB-PREX23-PURL-gpo193638>

td2124-crypto. (2026). unicc [GitHub repository]. GitHub. <https://github.com/td2124-crypto/unicc>

td2124-crypto. (2026). README.md. <https://raw.githubusercontent.com/td2124-crypto/unicc/main/README.md>

td2124-crypto. (2026). Architecture notes. <https://raw.githubusercontent.com/td2124-crypto/unicc/main/docs/architecture.md>

td2124-crypto. (2026). Agent application source.

<https://raw.githubusercontent.com/td2124-crypto/unicc/main/agents/app.py>

td2124-crypto. (2026). Orchestrator application source.

<https://raw.githubusercontent.com/td2124-crypto/unicc/main/orchestrator/app.py>

td2124-crypto. (2026). Demo output. https://raw.githubusercontent.com/td2124-crypto/unicc/main/demo_output.json

Appendix D. Other Supporting Documents

Supporting technical materials available in the repository include configuration files for each agent, prompt files, launch scripts, a sample payload, and a recorded demo output.

These materials support the claims made in the body of the report regarding system architecture, implementation scope, and use-case demonstration.

Recommended items to include here for final submission, if available, are screenshots of the dashboard, architecture diagram images, selected code excerpts, testing notes, and any sponsor-facing summary or handoff materials produced during the project.